

Fundamentos De Proximidad

Arlet Pinacho Báez Isaac Giovani Escobar González
Kevin Jonathan Garduño Escobar Seldon Sautto Ramírez

Invalid Date

Table of contents

Introducción	5
1 Fundamentos teóricos de proximidad	6
1.1 Introducción	6
2 Conceptos fundamentales de proximidad	7
2.1 La proximidad	7
2.2 Similitud	7
2.2.1 Propiedades características	7
2.3 Disimilitud	8
2.3.1 Propiedades características	8
2.4 Transformación entre similitud y disimilitud	9
3 Métricas: La Formalización Matemática de la Distancia	10
3.1 ¿Qué es una métrica?	10
3.2 Las Tres Propiedades de una Métrica	10
3.2.1 Propiedad 1: Positividad	10
3.2.2 Propiedad 2: Simetría	11
3.2.3 Propiedad 3: Desigualdad triangular	11
3.3 ¿Por qué importan?	12
3.4 Medidas que No Son Métricas	12
3.4.1 Diferencia de conjuntos (asimétrica)	12
3.4.2 Similitud del coseno (no es distancia directamente)	13
4 Similitud por Tipo de Atributo	14
4.1 Clasificación de atributos	14
4.2 Tabla de proximidad por tipo de atributo	14
5 Caso de Aplicación	15
5.1 Contexto: Segmentación de Clientes Bancarios	15
5.1.1 Paso 1: Identificar el tipo de cada atributo	15
5.1.2 Paso 2: Calcular similitud por atributo	15
5.1.3 Paso 3: Combinar	16
5.2 Preguntas de Reflexión	16

6	Distancias	17
6.1	Distancia Euclideana	17
6.1.1	Limitaciones	18
6.2	Distancia Manhattan	21
6.2.1	Limitaciones	23
6.3	Distancia de Hamming	24
6.3.1	Limitaciones	25
6.4	Distancia de Gower	25
6.5	Preguntas de reflexión	27
7	Similitud Coseno	29
7.1	Introducción Teórica	29
7.2	¿Qué es un Vector?	30
7.3	Fórmulas	30
7.3.1	Producto Punto	31
7.3.2	Magnitud del Vector	31
7.4	Procedimiento de Cálculo	31
7.5	Ejemplo Aplicado (Comparación de Personas)	32
7.5.1	Similitud entre A y B	32
7.5.2	Similitud entre A y C	33
7.5.3	Similitud entre B y C	34
7.6	Conclusión del Ejemplo	34
7.7	Distancia Coseno	36
7.8	Casos de Aplicación	37
7.8.1	1. Sistemas de recomendación	37
7.8.2	2. Procesamiento de texto	37
7.8.3	3. Reconocimiento facial	37
7.9	¿Cuándo conviene usar Similitud Coseno?	37
7.10	Conclusión	38
7.11	Preguntas de Reflexión	38
8	El Problema de las Diferentes Escalas	39
8.1	Pequeño ejemplo: Impacto de la escala en la distancia euclídea	39
8.2	Técnicas de Escalamiento	40
8.2.1	Normalización Min-Max	41
8.2.2	Estandarización Z-score	43
8.3	Impacto Práctico: Análisis Comparativo	45
8.3.1	Tabla comparativa de distancias	45
8.3.2	Visualización del espacio geométrico y la distorsión	45
8.3.3	Interpretación	48
8.4	Resumen Comparativo: Min-Max vs. Z-score	49
8.5	Preguntas de Reflexión	49

Introducción

En este documento se desarrollan los fundamentos de proximidad en minería de datos, incluyendo medidas de distancia y similitud, así como técnicas de escalamiento y normalización.

1 Fundamentos teóricos de proximidad

1.1 Introducción

En el análisis de datos, con frecuencia necesitamos agrupar o clasificar objetos basándonos en sus características. Entonces, necesitamos medidas cuantitativas que nos indiquen qué tan relacionados están dos puntos en nuestro espacio de datos.

Pero antes de agrupar, clasificar o detectar anomalías en datos, cualquier algoritmo de minería necesita responder una pregunta elemental:

¿Qué tan parecidos o diferentes son dos objetos de datos?

Esto nos lleva al concepto de **proximidad**, entenderlo es esencial para que podamos comprender mejor algunas técnicas como el *clustering*.

2 Conceptos fundamentales de proximidad

2.1 La proximidad

En minería de datos, la **proximidad** es una medida que cuantifica qué tan similares o diferentes son dos objetos de datos. Es por esto que la *proximidad* agrupa dos conceptos complementarios:

$$\text{Proximidad} \begin{cases} \textbf{Similitud} & \text{qué tan parecidos son dos objetos} \\ \textbf{Disimilitud} & \text{qué tan diferentes son dos objetos} \end{cases}$$

Así para entender mejor este concepto, es necesario entender ambos términos, además la elección entre usar similitud o disimilitud depende del algoritmo y del contexto, pero ambas pueden transformarse una en la otra.

2.2 Similitud

La **similitud** entre dos objetos x e y es una medida numérica que indica el grado en que ambos objetos se parecen. Se denota $s(x, y)$.

2.2.1 Propiedades características

1. $s(x, y) \in [0, 1]$ (no negativa y acotada)
2. $s(x, y) = 1 \iff x = y$ (máxima similitud solo con objetos idénticos)
3. $s(x, y) = s(y, x)$ (simetría)

Donde $s = 0$ indica **ninguna similitud** y $s = 1$ indica **similitud completa**.

💡 Nota para entender mejor

En el contexto de reconocimiento facial, la similitud se utiliza para comparar dos imágenes de rostros y determinar qué tan parecidos son. Por ejemplo, si tenemos dos imágenes de la misma persona, la similitud entre ellas debería ser alta (cercana a 1). Por otro lado, si

las imágenes son de personas diferentes, la similitud debería ser baja (cercana a 0).

2.3 Disimilitud

La **disimilitud** entre dos objetos x e y es una medida numérica del grado en que ambos objetos difieren. Se denota $d(x, y)$.

2.3.1 Propiedades características

A la inversa de la similitud, las disimilitudes arrojan valores más bajos para pares de objetos que son más similares.

1. $d(x, y) \geq 0$ (no negativa)
2. $d(x, y) = 0 \iff x = y$ (cero solo para objetos idénticos)
3. $d(x, y) = d(y, x)$ (simetría)

El rango de la disimilitud puede ser $[0, 1]$ o $[0, \infty)$ dependiendo de la medida utilizada.

💡 Nota para entender mejor

En el contexto de reconocimiento facial, la disimilitud se utiliza para comparar dos imágenes de rostros y determinar qué tan diferentes son. Por ejemplo, si tenemos dos imágenes de la misma persona, la disimilitud entre ellas debería ser baja (cercana a 0). Por otro lado, si las imágenes son de personas diferentes, la disimilitud debería ser alta (cercana a 1 o incluso mayor dependiendo de la medida utilizada).

❗ Terminología

Algunas veces nos encontramos con que **distancia** se usa como sinónimo de disimilitud. Sin embargo, técnicamente la distancia es un tipo especial de disimilitud que cumple propiedades adicionales. Lo que nos lleva a que no son lo mismo.

2.4 Transformación entre similitud y disimilitud

Dado que la similitud y la disimilitud son conceptos complementarios, es posible transformar una medida de similitud en una medida de disimilitud y viceversa. Esto es útil porque algunos algoritmos requieren una medida de similitud, mientras que otros requieren una medida de disimilitud. Si ambas están normalizadas en $[0, 1]$, la transformación más directa es:

$$\boxed{d(x, y) = 1 - s(x, y)} \quad \Longleftrightarrow \quad \boxed{s(x, y) = 1 - d(x, y)}$$

Existen otras transformaciones cuando los rangos no son $[0, 1]$. Por ejemplo, para convertir una disimilitud con rango $[0, \infty)$ a similitud en $[0, 1]$:

$$s(x, y) = \frac{1}{1 + d(x, y)} \quad \text{o} \quad s(x, y) = e^{-d(x, y)}$$

Este concepto es muy importante porque sin una medida matemática de proximidad, una computadora sería incapaz de encontrar patrones o relaciones entre los datos.

3 Métricas: La Formalización Matemática de la Distancia

Como ya vimos en la sección anterior, frecuentemente se suele usar la palabra **distancia** como sinónimo de disimilitud. Sin embargo, en el análisis de datos, para que una medida de disimilitud sea considerada como una métrica, debe satisfacer tres propiedades matemáticas bien definidas. Pero primero veamos la definición formal de una métrica.

3.1 ¿Qué es una métrica?

Una **métrica** es una función de disimilitud $d : X \times X \rightarrow \mathbb{R}$ que satisface **tres propiedades fundamentales**.

3.2 Las Tres Propiedades de una Métrica

Sea $d(x, y)$ la distancia entre dos objetos x e y en un conjunto X , las propiedades que se deben cumplir son las siguientes.

3.2.1 Propiedad 1: Positividad

Esta propiedad se divide en dos partes:

1. **No negatividad:**

$$d(x, y) \geq 0 \quad \forall x, y \in X$$

Las distancias nunca pueden ser negativas. Una distancia negativa no tiene interpretación física ni matemática coherente.

2. **Identidad del indiscernible:**

$$d(x, y) = 0 \iff x = y$$

La distancia entre un objeto y sí mismo es cero, **únicamente** entre un objeto y sí mismo.

3.2.2 Propiedad 2: Simetría

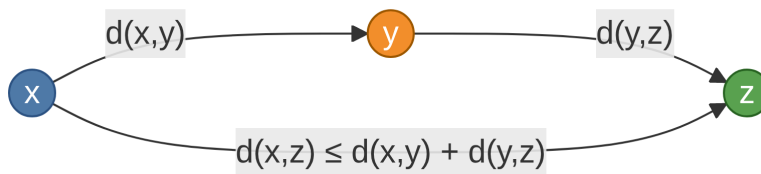
$$d(x, y) = d(y, x) \quad \forall x, y \in X$$

La distancia de x a y es exactamente igual que la distancia de y a x .

3.2.3 Propiedad 3: Desigualdad triangular

$$d(x, z) \leq d(x, y) + d(y, z) \quad \forall x, y, z \in X$$

La distancia directa entre dos puntos siempre será menor o igual a la distancia calculada si pasamos por un tercer punto intermedio. El siguiente diagrama ilustra esta propiedad geoméricamente:



Desigualdad triangular: el camino directo de x a z nunca puede ser mayor que pasar por y .

🔥 Pseudométricas

Existen medidas de disimilitud que no cumplen todas las propiedades de una métrica, estas se conocen como **pseudométricas**. Por ejemplo, algunas medidas cumplen $d(x, x) = 0$ pero **no** la dirección inversa: puede ocurrir que $d(x, y) = 0$ con $x \neq y$.

🔥 Asimetría

Si una medida de disimilitud no cumple la propiedad de simetría, se dice que es **asimétrica**. Un ejemplo de asimetría útil es: La distancia-tiempo entre dos horarios del día: si un evento ocurre a la 1 PM cada día y ahora son las 2 PM, esperas 1 hora; pero si estás a la 1 PM, el próximo evento hace 23 horas que ocurrió. Esta medida es útil, pero no es simétrica y por tanto no es una métrica.

3.3 ¿Por qué importan?

Importan porque muchas técnicas de análisis de datos, como el clustering, la clasificación o la detección de anomalías, se basan en la idea de proximidad entre objetos. Si la medida de disimilitud que usamos no es una métrica, podríamos obtener resultados inconsistentes o difíciles de interpretar.

En cambio, si usamos una métrica, podemos confiar en que las distancias reflejan relaciones coherentes entre los objetos, lo que facilita la interpretación y el análisis de los datos. Cada propiedad de una métrica nos garantiza algo en concreto:

Propiedad	Fórmula	¿Qué garantiza?
No negatividad	$d(x, y) \geq 0$	Distancias siempre positivas
Identidad	$d(x, y) = 0 \iff x = y$	Solo objetos idénticos tienen distancia cero
Simetría	$d(x, y) = d(y, x)$	La distancia no depende del sentido
Desig. triangular	$d(x, z) \leq d(x, y) + d(y, z)$	Consistencia geométrica del espacio

3.4 Medidas que No Son Métricas

No toda medida de disimilitud o similitud útil en minería de datos es una métrica. Como por ejemplo:

3.4.1 Diferencia de conjuntos (asimétrica)

Dados dos conjuntos A y B , definir $d(A, B) = |A - B|$ (cardinalidad de la diferencia) **no** es una métrica porque:

- No es simétrica: $|A - B| \neq |B - A|$ en general.

La versión simétrica $d(A, B) = |A - B| + |B - A|$ sí es una métrica.

3.4.2 Similitud del coseno (no es distancia directamente)

La similitud coseno $s_{\cos}(x, y) \in [-1, 1]$ tampoco satisface directamente las propiedades métricas, porque por ejemplo, no cumple la propiedad de la no negatividad ya que puede tomar valores negativos, aunque su transformación $d = 1 - s_{\cos}$ sí puede convertirse en una métrica bajo ciertas condiciones.

4 Similitud por Tipo de Atributo

La forma de calcular la similitud o disimilitud entre dos objetos depende del **tipo de atributo** que los describe.

4.1 Clasificación de atributos

Tipo	Ejemplo	Operaciones válidas
Nominal	color, género, país	$=, \neq$
Ordinal	{malo, regular, bueno}	$=, \neq, <, >$
Intervalo	temperatura en °C	$=, \neq, <, >, +, -$
Razón	peso, altura, precio	$=, \neq, <, >, +, -, \times, /$

4.2 Tabla de proximidad por tipo de atributo

Tipo	Disimilitud $d(x, y)$	Similitud $s(x, y)$
Nominal	$d = \begin{cases} 0 & x = y \\ 1 & x \neq y \end{cases}$	$s = \begin{cases} 1 & x = y \\ 0 & x \neq y \end{cases}$
Ordinal	$d = \frac{\ x - y\ }{n - 1}$	$s = 1 - d$
Inter- valo/Razón	$d = \ x - y\ $	$s = -d, s = \frac{1}{1 + d} \text{ o } e^{-d}$

Para atributos **ordinales**, los valores se mapean primero a enteros $\{0, 1, \dots, n - 1\}$ para capturar el orden.

5 Caso de Aplicación

5.1 Contexto: Segmentación de Clientes Bancarios

Una institución bancaria quiere agrupar a sus clientes para diseñar paquetes de productos personalizados. Cada cliente se describe con los atributos:

Atributo	Tipo	Ejemplo
Edad	Razón	34 años
Ingreso mensual	Razón	\$15,000 MXN
Categoría de cuenta	Nominal	“Premier”, “Básica”
Nivel de riesgo	Ordinal	{bajo, medio, alto}

5.1.1 Paso 1: Identificar el tipo de cada atributo

Primero hay que conocer el tipo de cada atributo para elegir la medida de proximidad correcta.

5.1.2 Paso 2: Calcular similitud por atributo

Para dos clientes a y b :

Edad: Si $\text{edad}_a = 34$, $\text{edad}_b = 38$:

$$d_{\text{edad}} = |34 - 38| = 4$$

Categoría de cuenta: Si a tiene “Premier” y b tiene “Básica”:

$$d_{\text{cuenta}} = 1 \quad (x \neq y)$$

Nivel de riesgo ($n = 3$ valores: $\{0, 1, 2\}$): Si a es “bajo” (0) y b es “alto” (2):

$$d_{\text{riesgo}} = \frac{|0 - 2|}{3 - 1} = \frac{2}{2} = 1$$

5.1.3 Paso 3: Combinar

La similitud global se calcula como el promedio de similitudes individuales (normalizadas a $[0, 1]$). Es aquí donde se aplican las métricas de distancia que se verán en las siguientes secciones.

5.2 Preguntas de Reflexión

1. ¿Por qué es importante que una medida de disimilitud cumpla las propiedades de una métrica? ¿Qué problemas podrían surgir si usamos una medida que no es una métrica?
2. Como vimos, la similitud nominal es estrictamente binaria (0 o 1). Pero, ¿en un problema de la vida real se puede decir que la disimilitud entre el color “Rojo” y “Azul” es exactamente la misma que entre “Rojo” y “Naranja”? y ¿Por qué?
3. ¿En qué situaciones podría ser útil usar una medida de proximidad que no es una métrica?

6 Distancias

6.1 Distancia Euclideana

Sean a y b dos objetos con p atributos numéricos, su distancia euclidiana se calcula mediante la fórmula (López et al. 2021):

$$d(a, b) = \sqrt{(x_{a1} - x_{b1})^2 + (x_{a2} - x_{b2})^2 + \dots + (x_{ap} - x_{bp})^2}$$

Esta distancia es de las más conocidas pues **representa el camino más corto** entre dos puntos en el espacio euclidean.

Utilizando las funciones de NumPy se puede implementar como se muestra a continuación.

```
import numpy as np

# función
def distancia_euclideana(a, b):
    return np.sqrt(np.sum((np.array(a) - np.array(b))**2))

# ejemplo
obj_a = (1, 2)
obj_b = (4, 6)
distancia_2d = distancia_euclideana(obj_a, obj_b)
print(f"Distancia euclideana entre {obj_a} y {obj_b}: {distancia_2d:.2f}")
```

Distancia euclideana entre (1, 2) y (4, 6): 5.00

O bien, es posible utilizar la versión ya implementada y optimizada de SciPy.

```
from scipy.spatial.distance import euclidean

distancia_scipy = euclidean(obj_a, obj_b)
print(f"Distancia euclideana entre {obj_a} y {obj_b}: {distancia_scipy:.2f}")
```

Distancia euclideana entre (1, 2) y (4, 6): 5.00

6.1.1 Limitaciones

Si bien la distancia euclídeana es muy útil gracias a que es fácil de entender y nos garantiza el camino más corto, tiene algunas limitaciones importantes, como lo son las siguientes (Datacamp 2024).

Sensibilidad a la escala

Para ver como la escala en la que se representan nuestros datos es fundamental para el cálculo de esta medida de distancia veremos un ejemplo.

Supongamos que tenemos tres clientes, de los cuales conocemos su edad y su ingreso mensual en pesos mexicanos.

```
# clientes: (edad, ingreso_mensual)
cliente_a = (25, 8_000)
cliente_b = (26, 82_000)
cliente_c = (55, 9_000)
```

Y queremos ver que cliente se parece más al cliente A.

Ocupando nuestra función `distancia_euclídeana` implementada anteriormente, calculamos su distancia y obtenemos:

```
print("--- Sin escalar ---")
print(f"Distancia entre A y B: {distancia_euclídeana(cliente_a, cliente_b):.1f}")
print(f"Distancia entre A y C: {distancia_euclídeana(cliente_a, cliente_c):.1f}")
```

```
--- Sin escalar ---
Distancia entre A y B: 74000.0
Distancia entre A y C: 1000.4
```

Con estos resultados podemos ver que el cliente más cercano al cliente A es el cliente C, aún cuando la diferencia en edad es de 30 años. Entonces, si escalamos nuestros datos, restando la media y dividiendo entre la desviación estándar tenemos lo siguiente.

```
datos = np.array([cliente_a, cliente_b, cliente_c], dtype=float)

media = datos.mean(axis=0)
std = datos.std(axis=0)
```

```

datos_escalados = (datos - media) / std

a_s, b_s, c_s = datos_escalados

print("--- Con Z-Score ---")
print(f"Distancia entre A y B: {distancia_euclidea(a_s, b_s):.3f}")
print(f"Distancia entre A y C: {distancia_euclidea(a_s, c_s):.3f}")

```

```

--- Con Z-Score ---
Distancia entre A y B: 2.137
Distancia entre A y C: 2.157

```

Ahora los resultados han cambiado pues vemos que el cliente más cercano al cliente A es en realidad el cliente B.

Pero, ¿cómo sabemos que el segundo resultado es el que esperamos?. Bueno, al **no escalar** lo que tenemos es un rango en las edades de $55 - 25 = 30$ mientras que en los ingresos se tiene un rango de $8,2000 - 8,000 = 74,000$ por lo que al obtener la diferencia entre los atributos de los clientes, el valor a obtener puede ser mucho mayor para el ingreso mensual que para la edad y al elevarlo al cuadrado, se vuelve más evidente que **la diferencia de ingreso pesa mucho más que una diferencia en la edad**. Por otro lado, al aplicar **Z-score** lo que hacemos es **restar la media para obtener cuánto se aleja del valor promedio** y al **dividir se obtiene cuántas desviaciones estándar representa** esa distancia del valor promedio. De forma que ambas variables se miden en desviaciones estándar, y **la diferencia de ingreso pesa exactamente lo mismo que una diferencia en la edad**.

Sensibilidad a valores atípicos

Para esta segunda limitación, supongamos que tenemos un conjunto de datos en el que se registran las características de 4 casas.

```

# casas: [m2, habitaciones, antigüedad_años, precio]
casa_a = (80, 3, 10, 1_200_000)
casa_b = (82, 3, 11, 1_250_000)
casa_c = (81, 3, 10, 15_000_000)
casa_d = (200, 10, 5, 1_200_000)

```

En las tres primeras el número de metros cuadrados es bastante similar pues el rango es de 2 m^2 , el número de habitaciones es el mismo y el rango de la antigüedad es de tan solo 1 año. Además, el precio de las casas A y B es muy similar, pero la casa C tiene un diferencia de al menos \$13,750,000 con respecto a las otras.

Por otro lado, la última casa es distinta en metros cuadrados, habitaciones y antigüedad pero igual en precio que A.

Al calcular su distancia euclidiana obtenemos los siguientes resultados.

```
print(f"Distancia entre A y B: {distancia_euclideana(casa_a, casa_b):>12.1f}")
print(f"Distancia entre A y C: {distancia_euclideana(casa_a, casa_c):>12.1f}")
print(f"Distancia entre A y D: {distancia_euclideana(casa_a, casa_d):>12.1f}")
```

```
Distancia entre A y B:      50000.0
Distancia entre A y C:    13800000.0
Distancia entre A y D:      120.3
```

En ellos, vemos que la diferencia de la casa A a la casa C es abismal en comparación con las casas B y D, lo que es cierto si solo nos fijamos en el precio, pero **poco informativo si queremos tomar en cuenta que en el resto de los atributos las casas A, B y C son muy similares pero con D no lo son.**

Al normalizar los datos, los resultados cambian y vemos mejor esta relación.

```
datos = np.array([casa_a, casa_b, casa_c, casa_d], dtype=float)

media = datos.mean(axis=0)
std = datos.std(axis=0)
std_seguro = np.where(std == 0, 1, std)

datos_escalados = (datos - media) / std_seguro

a_s, b_s, c_s, d_s = datos_escalados

print("--- Con Z-Score ---")
print(f"Distancia entre A y B: {distancia_euclideana(a_s, b_s):.3f}")
print(f"Distancia entre A y C: {distancia_euclideana(a_s, c_s):.3f}")
print(f"Distancia entre A y D: {distancia_euclideana(a_s, d_s):.3f}")
```

```
--- Con Z-Score ---
Distancia entre A y B: 0.428
Distancia entre A y C: 2.312
Distancia entre A y D: 3.912
```

6.2 Distancia Manhattan

Sean a y b dos objetos con p atributos numéricos, su distancia Manhattan se calcula mediante la fórmula (López et al. 2021):

$$d(a, b) = |x_{a1} - x_{b1}| + |x_{a2} - x_{b2}| + \dots + |x_{ap} - x_{bp}|$$

En python podemos definirla como se muestra a continuación.

```
def distancia_manhattan(a, b):  
    return np.sum(np.abs(np.array(a) - np.array(b)))
```

También, es posible utilizar la versión ya implementada de SciPy.

```
from scipy.spatial.distance import cityblock  
  
a = (1, 1)  
b = (4, 5)  
  
dis_man_scipy = cityblock(a, b)  
print(f"La distancia Manhattan entre {a} y {b} es de {dis_man_scipy}")
```

La distancia Manhattan entre (1, 1) y (4, 5) es de 7

A diferencia de la distancia euclideana, la distancia Manhattan no mide el camino más corto entre los puntos, pues al sumar la diferencia absoluta atributo a atributo lo que mide, más intuitivamente, es el camino que recorrería un taxi en una ciudad cuadrículada.

```
import matplotlib.pyplot as plt  
  
dis_euc = distancia_euclideana(a, b)  
dis_man = distancia_manhattan(a, b)  
  
fig, ax = plt.subplots(figsize=(5, 5))  
  
# Euclídeana  
ax.plot([a[0], b[0]], [a[1], b[1]], color="#534AB7", linewidth=2.5,  
        label=f"Euclídea = {dis_euc:.2f}")  
  
# Manhattan
```

```

ax.plot([a[0], b[0], b[0]], [a[1], a[1], b[1]], color="#ba177b", linewidth=2.5,
        linestyle="--", label=f"Manhattan = {dis_man:.2f}")

# Anotaciones de los segmentos Manhattan
ax.annotate("", xy=(b[0], a[1]), xytext=(a[0], a[1]),
            arrowprops=dict(arrowstyle="->", color="#ba177b", lw=1.2))
ax.text((a[0] + b[0]) / 2, a[1] - 0.3, f" $\Delta x = \{b[0]-a[0]\}$ ",
        ha="center", color="#ba177b", fontsize=10)

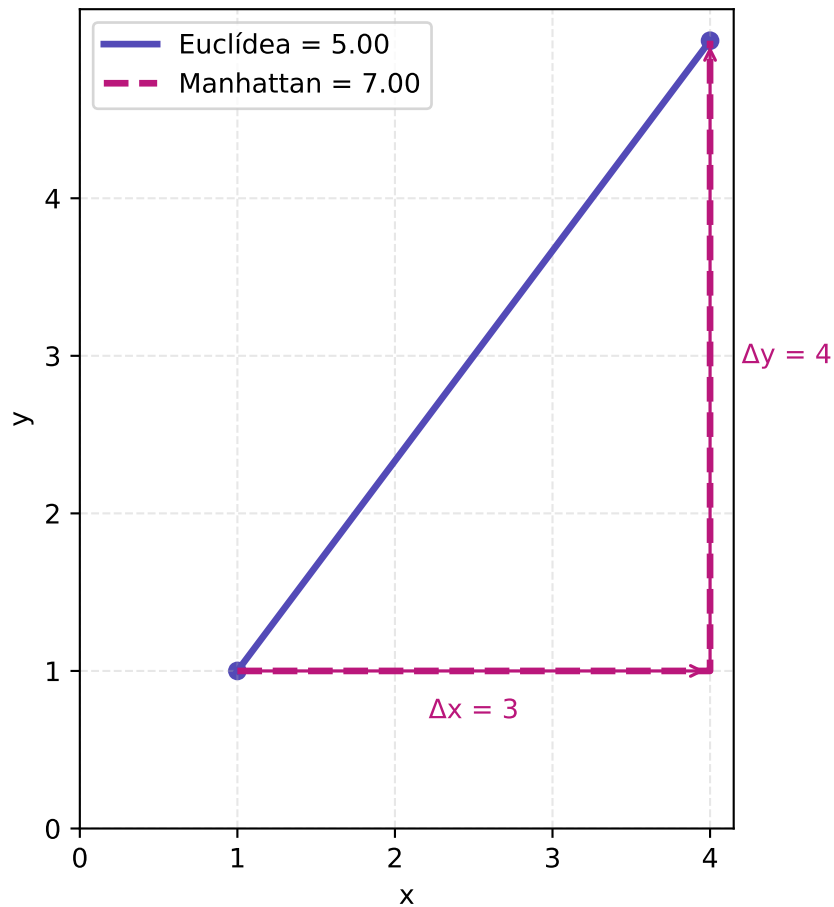
ax.annotate("", xy=(b[0], b[1]), xytext=(b[0], a[1]),
            arrowprops=dict(arrowstyle="->", color="#ba177b", lw=1.2))
ax.text(b[0] + 0.2, (a[1] + b[1]) / 2, f" $\Delta y = \{b[1]-a[1]\}$ ",
        va="center", color="#ba177b", fontsize=10)

# Puntos
ax.scatter(*a, color="#534AB7")
ax.scatter(*b, color="#534AB7")

ax.set_xticks(range(5))
ax.set_yticks(range(5))

ax.grid(True, linestyle="--", alpha=0.3)
ax.set_xlabel("x")
ax.set_ylabel("y")
ax.legend(loc="upper left")
ax.set_aspect("equal")
plt.tight_layout()
plt.show()

```



6.2.1 Limitaciones

La distancia Manhattan también tiene limitaciones como la sensibilidad a la escala y a la presencia de outliers, pero debido a que no eleva al cuadrado la diferencia de los atributos es significativamente más resistente (Medium 2026). Hay que tener en cuenta que esto no quiere decir que los valores obtenidos sean menores, pues como se mencionó con anterioridad, la distancia euclideana es la menor.

Veamos un ejemplo.

```
a = (1, 1, 1, 1, 1, 1, 1, 1, 1, 1)
b = (1, 1, 1, 1, 1, 1, 1, 1, 1, 21)
c = (2, 2, 2, 2, 2, 2, 2, 2, 2, 2)

print(f"Distancia Manhattan casa A y B: {distancia_manhattan(a, b)}")
```

```
print(f"Distancia Manhattan casa A y C: {distancia_manhattan(a, c)}")

print(f"Distancia Euclideana casa A y B: {distancia_euclideana(a, b)}")
print(f"Distancia Euclideana casa A y C: {distancia_euclideana(a, c)}")
```

```
Distancia Manhattan casa A y B: 20
Distancia Manhattan casa A y C: 10
Distancia Euclideana casa A y B: 20.0
Distancia Euclideana casa A y C: 3.1622776601683795
```

Notemos que con los valores de B y C originales, podemos ver que ambos son bastante similares. Sin embargo, tomando en cuenta solo las distancias obtenidas, tenemos una distancia en Manhattan de 10 puntos, mientras que en la Euclideana una distancia de casi 17 puntos. Esto refleja que la diferencia (outlier) en el último atributo tiene mucho más peso en esta segunda distancia.

6.3 Distancia de Hamming

Sean a y b dos objetos con p atributos categóricos, su distancia de Hamming se calcula mediante la fórmula (López et al. 2021):

$$d(a, b) = (x_{a1} \neq x_{b1}) + (x_{a2} \neq x_{b2}) + \dots + (x_{ap} \neq x_{bp})$$

Cada condición se evalúa de forma booleana, por lo que cada término es 1 o 0.

Se puede implementar como se muestra a continuación.

```
#función
def distancia_hamming(a, b):
    return np.sum(np.array(a) != np.array(b))
```

O bien se puede usar la versión de SciPy pero hay que tomar en cuenta que esta añade una división al final entre el número de atributos del objeto para normalizar el resultado (Datacamp 2025).


```

from scipy.spatial.distance import hamming

x = [1, 0, 1, 1, 0]
y = [1, 1, 1, 0, 0]

print(f"Hamming (propia): {distancia_hamming(x, y):.2f}")
print(f"Hamming (propia normalizada): {distancia_hamming(x, y)/len(x):.2f}")
print(f"Hamming SciPy: {hamming(x, y):.2f}")

```

```

Hamming (propia): 2.00
Hamming (propia normalizada): 0.40
Hamming SciPy: 0.40

```

6.3.1 Limitaciones

Algunas limitaciones importantes de este tipo de distancia, que hay que tomar en cuenta al momento de elegir, son por un lado su diseño orientado a la comparación de atributos categóricos y por el otro la dependencia a que ambos objetos tengan la misma cantidad de atributos (Datacamp 2025).

6.4 Distancia de Gower

Sean a y b dos objetos con p atributos mixtos, su distancia de Gower se calcula mediante la fórmula (López et al. 2021):

$$d(a, b) = \sum_{i=1}^p \delta(x_{ai}, x_{bi})$$

$$\delta(x_{ai}, x_{bi}) = \begin{cases} 1 & \text{si } x_{ai} \text{ y } x_{bi} \text{ son nominales y distintos.} \\ 0 & \text{si } x_{ai} \text{ y } x_{bi} \text{ son nominales e iguales.} \\ \frac{|x_{ai} - x_{bi}|}{\text{rango}(i)} & \text{si } x_{ai} \text{ y } x_{bi} \text{ son numericos.} \end{cases}$$

Una opción de implementación para esta distancia es la que se muestra a continuación.

```
def distancia_gower(a, b, tipos, rangos):
    a, b = np.array(a, dtype=object), np.array(b, dtype=object)
    contribuciones = 0

    for i in range(len(a)):
        if tipos[i] == 'n':
            d = abs(float(a[i]) - float(b[i])) / rangos[i]
        else:
            d = 0 if a[i] == b[i] else 1
        contribuciones += d

    return contribuciones
```

Por su naturaleza para el manejo de datos mixtos, para su cálculo es necesario saber el tipo de variable al que corresponde cada atributo y el rango de aquellas que son numéricas, por lo que además de los dos objetos a evaluar, es necesaria una lista en la que se especifica por cada atributo, si es categórico (c) o numérico (n) y otra lista con los rangos de cada atributo (0 si es categórica).

```
#ejemplo
# mascotas: [peso_kg, especie]
mascota_a = [4.0, "gato"]
mascota_b = [6.0, "gato"]
mascota_c = [5.0, "perro"]

tipos = ['n', 'c']
rangos = [2.0, 0]

print(f"Distancia de Gower entre A y B:")
print(f"{distancia_gower(mascota_a, mascota_b, tipos, rangos):.3f}")
print(f"Distancia de Gower entre A y C:")
print(f"{distancia_gower(mascota_a, mascota_c, tipos, rangos):.3f}")
```

```
Distancia de Gower entre A y B:
1.000
Distancia de Gower entre A y C:
1.500
```

6.5 Preguntas de reflexión

1. Si la distancia Manhattan es la suma de diferencias absolutas, ¿qué forma geométrica describe el conjunto de todos los puntos equidistantes a un punto central? ¿Y en el caso de Euclídea?

```
import matplotlib.pyplot as plt

fig, axes = plt.subplots(1, 2, figsize=(12, 4))

theta = np.linspace(0, 2 * np.pi, 300)
r = 1.0

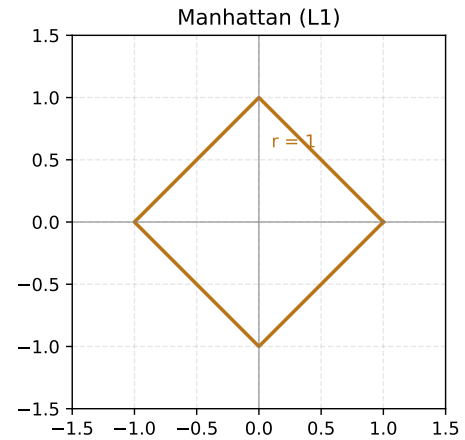
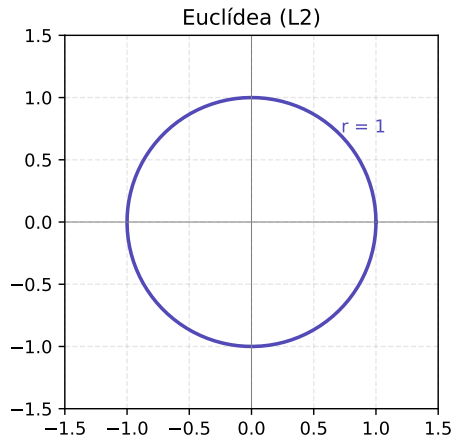
# Euclídeana: circunferencia
ax = axes[0]
x = r * np.cos(theta)
y = r * np.sin(theta)
ax.plot(x, y, color="#534AB7", linewidth=2)
ax.set_title("Euclídea (L2)", fontsize=12)
ax.set_aspect("equal")
ax.axhline(0, color="gray", linewidth=0.5)
ax.axvline(0, color="gray", linewidth=0.5)
ax.set_xlim(-1.5, 1.5)
ax.set_ylim(-1.5, 1.5)
ax.grid(True, linestyle="--", alpha=0.3)
ax.annotate("r = 1", xy=(0.72, 0.72), color="#534AB7", fontsize=10)

# Manhattan: rombo
ax = axes[1]
rombo_x = [1, 0, -1, 0, 1]
rombo_y = [0, 1, 0, -1, 0]
ax.plot(rombo_x, rombo_y, color="#BA7517", linewidth=2)
ax.set_title("Manhattan (L1)", fontsize=12)
ax.set_aspect("equal")
ax.axhline(0, color="gray", linewidth=0.5)
ax.axvline(0, color="gray", linewidth=0.5)
ax.set_xlim(-1.5, 1.5)
ax.set_ylim(-1.5, 1.5)
ax.grid(True, linestyle="--", alpha=0.3)
ax.annotate("r = 1", xy=(0.1, 0.6), color="#BA7517", fontsize=10)

plt.suptitle("Conjunto de puntos equidistantes según cada métrica", fontsize=13)
```

```
plt.tight_layout()
plt.show()
```

Conjunto de puntos equidistantes según cada métrica



- ¿Qué pasa en el caso de Gower si dos objetos son idénticos en todas las variables continuas pero difieren en una categórica?

```
# personas: [peso, estatura, edad, animal_favorito]
persona_a = [45.5, 1.6, 8, "gato "]
persona_b = [45.5, 1.6, 8, "gato"]

tipos = ['n', 'n', 'n', 'c']
rangos = [20, 0.7, 10, 0] # suponemos que se obtienen a partir de más datos

print(f"Distancia de Gower entre A y B:")
print(f"{distancia_gower(persona_a, persona_b, tipos, rangos):.2f}")
```

Distancia de Gower entre A y B:
1.00

- ¿Cómo se relaciona la multidimensionalidad con la distancia euclídea y la manhattan?

[ver más](#)

7 Similitud Coseno

7.1 Introducción Teórica

La **Similitud Coseno** es una operación matemática que permite cuantificar el grado de similitud entre dos vectores.

En términos simples, mide qué tan parecidos son dos conjuntos de datos representados numéricamente.

Mientras que otras medidas (como la distancia Euclídea) miden qué tan lejos están dos puntos, la similitud coseno mide qué tan alineados están en el espacio.

Es decir:

- No compara magnitud.
- Compara dirección.
- Mide el ángulo entre vectores.

Puede interpretarse como comparar flechas:

- Si apuntan en la misma dirección $\rightarrow$ son muy similares.
- Si forman un ángulo recto $\rightarrow$ no están relacionadas.
- Si apuntan en direcciones opuestas $\rightarrow$ son opuestas.

Matemáticamente, los valores oscilan entre -1 y 1:

- 1 $\rightarrow$ vectores exactamente en la misma dirección.
- 0 $\rightarrow$ vectores ortogonales (sin relación direccional).
- -1 $\rightarrow$ vectores en direcciones opuestas.

Esta medida es base fundamental de muchas aplicaciones de Machine Learning e Inteligencia Artificial, como:

- Sistemas de recomendación
- Reconocimiento facial
- Búsqueda de documentos
- Procesamiento de lenguaje natural

Muchos sistemas digitales modernos funcionan midiendo similitud.

7.2 ¿Qué es un Vector?

Un **vector** es una forma compacta de representar un dato numéricamente.

Por ejemplo, si representamos una persona con:

- Edad
- Peso
- Altura

Podemos escribir:

Persona A: [(25, 70, 1.75)]

Persona B: [(30, 80, 1.80)]

Cada característica es una dimensión del vector.

En reconocimiento facial, un embedding es simplemente una representación vectorial del rostro en muchas dimensiones.

7.3 Fórmulas

La **Similitud Coseno** entre dos vectores $\vec{a}$ y $\vec{b}$ se define como:

$$\text{similitud}(\vec{a}, \vec{b}) = \frac{\vec{a} \cdot \vec{b}}{||\vec{a}|| ||\vec{b}||}$$

Donde:

- $(\vec{a} \cdot \vec{b})$ es el **producto punto**.
- $(||\vec{a}||)$ es la **magnitud** del vector $\vec{a}$.
- $(||\vec{b}||)$ es la **magnitud** del vector $\vec{b}$.

7.3.1 Producto Punto

$$(\vec{a} \cdot \vec{b} = a_1b_1 + a_2b_2 + a_3b_3)$$

Se obtiene multiplicando las componentes correspondientes de cada vector y sumando los resultados.

7.3.2 Magnitud del Vector

$$(\|\vec{v}\| = \sqrt{x^2 + y^2 + z^2})$$

Se calcula elevando cada componente al cuadrado, sumando los valores y aplicando raíz cuadrada.

7.4 Procedimiento de Cálculo

1. **Calcular el producto punto de los vectores.**

Multiplicar las componentes correspondientes y sumar los resultados.

2. **Calcular la magnitud de cada vector.**

Obtener la raíz cuadrada de la suma de los cuadrados de sus componentes.

3. **Multiplicar las magnitudes de ambos vectores.**

4. **Dividir el producto punto entre el producto de las magnitudes.**

5. **Interpretar el resultado.**

El valor obtenido estará entre -1 y 1 y representa el grado de similitud direccional entre los vectores.

7.5 Ejemplo Aplicado (Comparación de Personas)

Supongamos que queremos medir qué tan similares son tres personas en términos de:

- Horas de estudio por semana
- Horas de ejercicio por semana

Representamos cada persona como un vector:

Persona A:

$$(10, 5)$$

Persona B:

$$(8, 4)$$

Persona C:

$$(2, 9)$$

La idea es cuantificar qué tan similares son sus estilos de vida.

7.5.1 Similitud entre A y B

Producto punto:

$$(10 \times 8) + (5 \times 4) = 80 + 20 = 100$$

Magnitudes:

$$||A|| = \sqrt{10^2 + 5^2} = \sqrt{100 + 25} = \sqrt{125}$$

$$||B|| = \sqrt{8^2 + 4^2} = \sqrt{64 + 16} = \sqrt{80}$$

Similitud:

$$\text{sim}(A, B) = \frac{100}{\sqrt{125} \times \sqrt{80}}$$

Aproximadamente:

$$\text{sim}(A, B) \approx 0.999$$

Interpretación:

A y B tienen proporciones muy similares entre estudio y ejercicio, por eso la similitud es casi perfecta.

7.5.2 Similitud entre A y C

Producto punto:

$$(10 \times 2) + (5 \times 9) = 20 + 45 = 65$$

Magnitudes:

$$\|C\| = \sqrt{2^2 + 9^2} = \sqrt{4 + 81} = \sqrt{85}$$

Similitud:

$$\text{sim}(A, C) = \frac{65}{\sqrt{125} \times \sqrt{85}}$$

Aproximadamente:

$$\text{sim}(A, C) \approx 0.63$$

Interpretación:

A y C no tienen la misma proporción entre estudio y ejercicio, por lo tanto la similitud disminuye.

7.5.3 Similitud entre B y C

Producto punto:

$$(8 \times 2) + (4 \times 9) = 16 + 36 = 52$$

Similitud:

$$\text{sim}(B, C) = \frac{52}{\sqrt{80} \times \sqrt{85}}$$

Aproximadamente:

$$\text{sim}(B, C) \approx 0.63$$

7.6 Conclusión del Ejemplo

- A y B son casi idénticos en proporción.
- A y C son moderadamente similares.
- B y C también muestran similitud moderada.

Este ejemplo muestra que la similitud coseno no compara cantidades absolutas, sino la dirección o proporción entre variables.

Es útil cuando queremos identificar patrones de comportamiento similares, aunque los valores no sean exactamente iguales.

A continuación se muestran los vectores que representan a cada persona en el plano (Estudio vs Ejercicio):

```
import matplotlib.pyplot as plt
import numpy as np

# Definir vectores
A = np.array([10, 5])
B = np.array([8, 4])
C = np.array([2, 9])

plt.figure(figsize=(6,6))
```

```

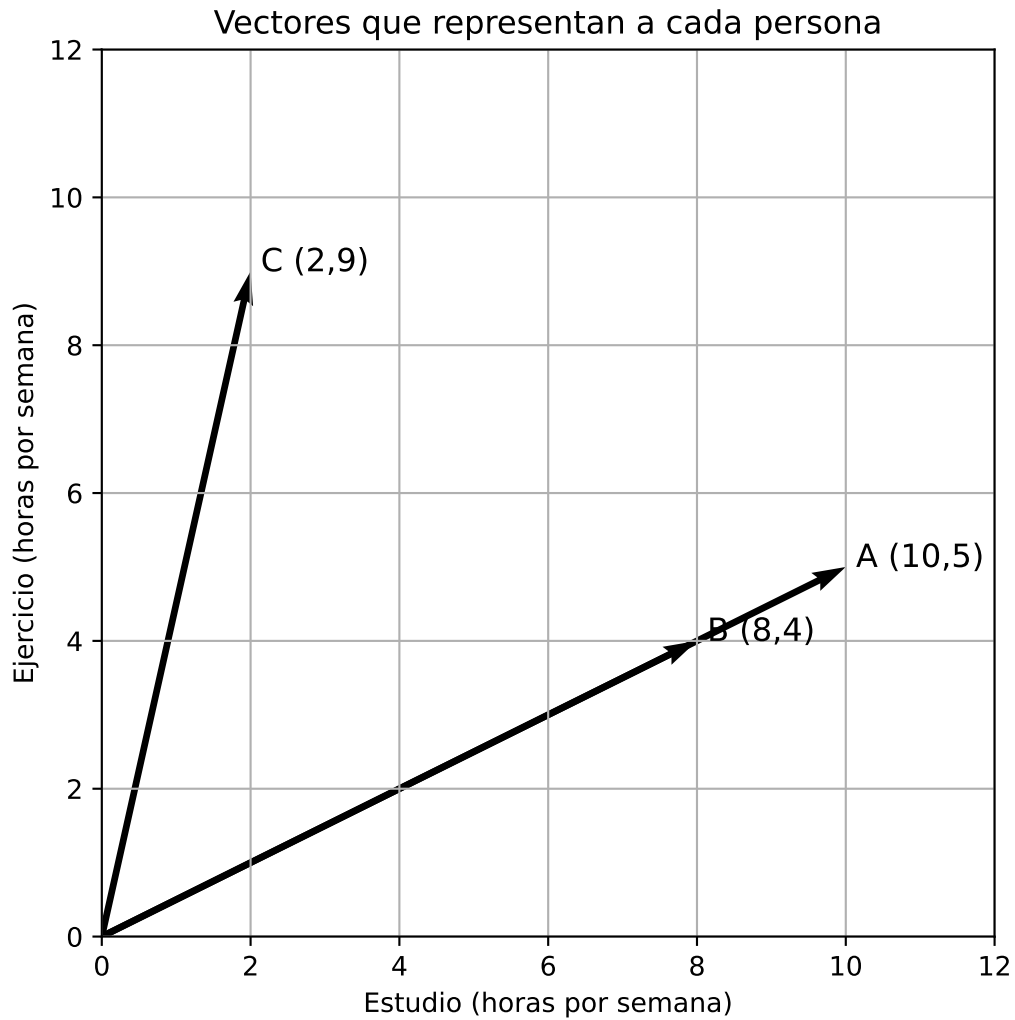
# Dibujar vectores desde el origen
plt.quiver(0, 0, A[0], A[1], angles='xy', scale_units='xy', scale=1)
plt.quiver(0, 0, B[0], B[1], angles='xy', scale_units='xy', scale=1)
plt.quiver(0, 0, C[0], C[1], angles='xy', scale_units='xy', scale=1)

# Etiquetas
plt.text(A[0], A[1], " A (10,5)", fontsize=12)
plt.text(B[0], B[1], " B (8,4)", fontsize=12)
plt.text(C[0], C[1], " C (2,9)", fontsize=12)

# Configuración del plano
plt.xlim(0, 12)
plt.ylim(0, 12)
plt.xlabel("Estudio (horas por semana)")
plt.ylabel("Ejercicio (horas por semana)")
plt.title("Vectores que representan a cada persona")
plt.grid(True)

plt.show()

```



7.7 Distancia Coseno

La **Distancia Coseno** se define como:

$$d_{cos}(A, B) = 1 - \text{Similitud Coseno}(A, B)$$

Se utiliza cuando un algoritmo requiere una **medida de distancia** en lugar de una medida de similitud, especialmente en métodos de clustering o modelos que trabajan con distancias.

Mientras menor sea la distancia coseno, mayor será la similitud entre los vectores.

7.8 Casos de Aplicación

7.8.1 1. Sistemas de recomendación

Se representan usuarios como vectores de preferencias y se comparan mediante similitud coseno para recomendar productos similares.

7.8.2 2. Procesamiento de texto

Cada documento se representa como un vector (TF-IDF). La similitud coseno mide qué tan similares son dos textos.

7.8.3 3. Reconocimiento facial

Cada rostro se transforma en un embedding (vector numérico). Se comparan vectores para determinar si pertenecen a la misma persona.

7.9 ¿Cuándo conviene usar Similitud Coseno?

Conviene usarla cuando:

- Importa la proporción, no la magnitud.
- Los datos tienen alta dimensionalidad.
- Se trabaja con texto o embeddings.
- Se quiere comparar patrones de comportamiento.

No es ideal cuando:

- La magnitud representa información importante.
 - Se requiere una distancia física real.
-

7.10 Conclusión

La similitud coseno es una herramienta poderosa para cuantificar semejanza entre vectores.

No mide distancia absoluta, sino alineación direccional.

Es fundamental en múltiples aplicaciones modernas de Machine Learning.

7.11 Preguntas de Reflexión

1. Si dos vectores tienen valores muy diferentes en magnitud pero mantienen la misma proporción, ¿qué resultado obtendríamos en la similitud coseno y qué significa geométricamente?
2. ¿Qué limitación tiene la similitud coseno cuando la magnitud de los datos sí representa información importante?
3. ¿En qué situaciones sería más apropiado utilizar similitud coseno en lugar de distancia Euclídea?

8 El Problema de las Diferentes Escalas

Al aplicar algoritmos de minería de datos que dependen de medidas de proximidad (como la **distancia euclídea**, la **distancia Manhattan** o la **similitud coseno**), nos enfrentamos a un problema fundamental: la magnitud de las variables.

Los datos deben estar escalados para evitar que algún atributo con un rango de valores mucho mayor que los otros predomine al momento de calcular las distancias. Si los atributos no comparten un marco de referencia comparable, el espacio geométrico se distorsiona: las variables numéricas con rangos más amplios **dominan artificialmente** el cálculo de distancia, minimizando o anulando por completo la contribución de otras variables que, aunque numéricamente menores, pueden ser igualmente relevantes para el problema.

Definición. El *problema de las escalas* surge cuando los atributos del conjunto de datos tienen rangos de variación heterogéneos, lo que provoca que las métricas de distancia queden sesgadas hacia los atributos de mayor magnitud numérica, independientemente de su relevancia para el problema.

8.1 Pequeño ejemplo: Impacto de la escala en la distancia euclídea

La distancia euclídea entre dos objetos a y b con p atributos se define como:

$$d(a, b) = \sqrt{\sum_{i=1}^p (x_{ai} - x_{bi})^2}$$

Para ilustrar el problema, considere el siguiente conjunto de datos bancario con los atributos **Salario mensual (MXN)** y **Edad (años)**:

	Salario (MXN/mes)	Edad (años)
Persona		
A	15000	25
B	15500	55
C	15200	40

Calculamos la distancia euclídea entre la Persona **A** y la Persona **B** sin ningún escalamiento previo:

$$\begin{aligned}
 d(A, B) &= \sqrt{(15,000 - 15,500)^2 + (25 - 55)^2} \\
 &= \sqrt{(-500)^2 + (-30)^2} \\
 &= \sqrt{250,000 + 900} \\
 &= \sqrt{250,900} \approx 500.9
 \end{aligned}$$

	Contrib. Salario	Contrib. Edad	Distancia Total	% Salario	% Edad
Par					
A — B	250,000	900	500.9	99.6 %	0.4 %
A — C	40,000	225	200.6	99.4 %	0.6 %
B — C	90,000	225	300.4	99.8 %	0.2 %

Análisis de la distorsión. El término del salario aporta **250,000** al cálculo bajo la raíz cuadrada, mientras que la edad aporta apenas **900**. Una diferencia de 30 años de vida (un cambio generacional completo) es matemáticamente aplastada por una diferencia de 500 pesos en el salario. El algoritmo percibe a las personas basándose casi exclusivamente en lo que ganan, ignorando por completo su edad.

Conclusión. Para que todos los atributos contribuyan de manera equitativa al cálculo de similitud, es indispensable transformarlos a una escala común antes de aplicar cualquier algoritmo de clustering o clasificación basado en distancias.

8.2 Técnicas de Escalamiento

El escalamiento (también denominado normalización o estandarización) es el proceso de transformar los valores de los atributos a un rango o distribución común, de modo que ningún atributo domine a los demás por razones puramente de magnitud. Las dos técnicas más utilizadas son:

1. **Normalización Min-Max** — proyecta los valores al intervalo $[0, 1]$.
2. **Estandarización Z-score** — centra los datos en media 0 con desviación estándar 1.

8.2.1 Normalización Min-Max

8.2.1.1 Fundamento teórico

La normalización Min-Max reescala cada atributo de forma **lineal** al intervalo $[0, 1]$. La transformación garantiza que el valor mínimo del atributo original se mapea a 0 y el máximo a 1:

$$x'_i = \frac{x_i - x_{\min}}{x_{\max} - x_{\min}}$$

donde $x_{\min}$ y $x_{\max}$ son el mínimo y el máximo del atributo en el conjunto de datos, y $x'_i \in [0, 1]$ siempre.

8.2.1.2 Desarrollo paso a paso

Identificamos los parámetros de cada atributo:

- Salario: $x_{\min} = 15,000$, $x_{\max} = 15,500$, Rango = 500
- Edad: $x_{\min} = 25$, $x_{\max} = 55$, Rango = 30

Persona A (15,000 ; 25):

$$\text{Salario}'_A = \frac{15,000 - 15,000}{500} = 0.000 \quad \text{Edad}'_A = \frac{25 - 25}{30} = 0.000 \quad \Rightarrow \quad \mathbf{A}' = (0.000, 0.000)$$

Persona B (15,500 ; 55):

$$\text{Salario}'_B = \frac{15,500 - 15,000}{500} = 1.000 \quad \text{Edad}'_B = \frac{55 - 25}{30} = 1.000 \quad \Rightarrow \quad \mathbf{B}' = (1.000, 1.000)$$

Persona C (15,200 ; 40):

$$\text{Salario}'_C = \frac{15,200 - 15,000}{500} = 0.400 \quad \text{Edad}'_C = \frac{40 - 25}{30} = 0.500 \quad \Rightarrow \quad \mathbf{C}' = (0.400, 0.500)$$

```

scaler_mm = MinMaxScaler()
X_mm      = scaler_mm.fit_transform(X)

df_mm = pd.DataFrame(X_mm, columns=["Salario Min-Max", "Edad Min-Max"],
                      index=personas)
df_mm.insert(0, "Salario Original", X[:, 0])
df_mm.insert(2, "Edad Original",    X[:, 1])
df_mm.round(3)

```

	Salario Original	Salario Min-Max	Edad Original	Edad Min-Max
A	15000	0.0	25	0.0
B	15500	1.0	55	1.0
C	15200	0.4	40	0.5

Recalculo de distancias tras Min-Max:

	Distancia Min-Max
Par	
A — B	1.4142
A — C	0.6403
B — C	0.7810

Ahora cada atributo contribuye equitativamente: la distancia entre A y B es la mayor (difieren tanto en salario como en edad), lo que resulta conceptualmente coherente.

8.2.1.3 Ventajas y desventajas

Aspecto	Detalle
Rango garantizado	Los valores siempre quedan en $[0, 1]$; facilita la interpretación
Preserva relaciones relativas	La transformación es estrictamente lineal
Sensible a valores atípicos	Un solo extremo comprime todos los demás en un rango muy pequeño
Dependiente del conjunto de entrenamiento	Nuevos datos fuera del rango original violan el intervalo $[0, 1]$

8.2.2 Estandarización Z-score

8.2.2.1 Fundamento teórico

La estandarización Z-score transforma los datos de modo que el atributo resultante tenga **media igual a 0** y **desviación estándar igual a 1**. Al escalar con media cero y desviación estándar igual a uno, se consigue que todas las variables tengan igual peso en su participación al agrupar los datos. El valor resultante z indica cuántas desviaciones estándar se aleja un dato de la media de su distribución:

$$z_i = \frac{x_i - \bar{x}}{\sigma}$$

donde $\bar{x} = \frac{1}{n} \sum x_i$ es la **media** del atributo y $\sigma = \sqrt{\frac{1}{n} \sum (x_i - \bar{x})^2}$ es la **desviación estándar poblacional**.

8.2.2.2 Desarrollo paso a paso

Parámetros del atributo Salario:

$$\bar{x}_{\text{sal}} = \frac{15000 + 15500 + 15200}{3} \approx 15233.33 \quad \sigma_{\text{sal}} \approx 205.48$$

Parámetros del atributo Edad:

$$\bar{x}_{\text{edad}} = \frac{25 + 55 + 40}{3} = 40 \quad \sigma_{\text{edad}} \approx 12.25$$

Persona A (15,000 ; 25):

$$z_{\text{sal},A} = \frac{15000 - 15233.33}{205.48} \approx -1.136 \quad z_{\text{edad},A} = \frac{25 - 40}{12.25} \approx -1.225 \quad \Rightarrow \quad \mathbf{A_z} = (-1.136, -1.225)$$

Persona B (15,500 ; 55):

$$z_{\text{sal},B} = \frac{15500 - 15233.33}{205.48} \approx 1.298 \quad z_{\text{edad},B} = \frac{55 - 40}{12.25} \approx 1.225 \quad \Rightarrow \quad \mathbf{B_z} = (1.298, 1.225)$$

Persona C (15,200 ; 40):

$$z_{\text{sal},C} = \frac{15200 - 15233.33}{205.48} \approx -0.162 \quad z_{\text{edad},C} = \frac{40 - 40}{12.25} = 0.000 \quad \Rightarrow \quad \mathbf{C_z} = (-0.162, 0.000)$$

```
scaler_z = StandardScaler()
X_z      = scaler_z.fit_transform(X)

df_z = pd.DataFrame(X_z, columns=["Salario Z-score", "Edad Z-score"],
                    index=personas)
df_z.insert(0, "Salario Original", X[:, 0])
df_z.insert(2, "Edad Original",    X[:, 1])
df_z.round(3)
```

	Salario Original	Salario Z-score	Edad Original	Edad Z-score
A	15000	-1.136	25	-1.225
B	15500	1.298	55	1.225
C	15200	-0.162	40	0.000

Recalculo de distancias tras Z-score:

	Distancia Z-score
Par	
A — B	3.4527
A — C	1.5644
B — C	1.9057

8.2.2.3 Ventajas y desventajas

Aspecto	Detalle
Robusto ante valores atípicos	La dispersión real (σ) evita que extremos colapsen el rango
Recomendado para normalidad	Preferido en PCA, regresión y algoritmos que asumen distribución gaussiana
Sin rango fijo	Los valores resultantes no están acotados; pueden superar ± 3
Requiere conservar parámetros	Con nuevos datos deben usarse la $\bar{x}$ y σ del entrenamiento

8.3 Impacto Práctico: Análisis Comparativo

8.3.1 Tabla comparativa de distancias

```
filas = []
for _, _, i, j in pares:
    d_orig = euclidean(X[i], X[j])
    d_mm = euclidean(X_mm[i], X_mm[j])
    d_z = euclidean(X_z[i], X_z[j])
    filas.append({
        "Par": f"{personas[i]} - {personas[j]}",
        "Sin escalamiento": f"{d_orig:.1f}",
        "Min-Max": f"{d_mm:.4f}",
        "Z-score": f"{d_z:.4f}",
    })

rankings = ["3° (mas lejanos)", "1° (mas cercanos)", "2°"]
for fila, r in zip(filas, rankings):
    fila["Ranking"] = r

pd.DataFrame(filas).set_index("Par")
```

	Sin escalamiento	Min-Max	Z-score	Ranking
Par				
A — B	500.9	1.4142	3.4527	3° (mas lejanos)
A — C	200.6	0.6403	1.5644	1° (mas cercanos)
B — C	300.4	0.7810	1.9057	2°

8.3.2 Visualización del espacio geométrico y la distorsión

```
colores = ["#E53935", "#1E88E5", "#43A047"]
etiquetas = ["A - B", "A - C", "B - C"]
x_pos = np.arange(len(etiquetas))
ancho = 0.35

mm_dist = [euclidean(X_mm[i], X_mm[j]) for _, _, i, j in pares]
z_dist = [euclidean(X_z[i], X_z[j]) for _, _, i, j in pares]
pct_sal = contrib_sal / total * 100
```

```

pct_edad = contrib_edad / total * 100

fig = plt.figure(figsize=(14, 9))
gs = fig.add_gridspec(2, 3, hspace=0.50, wspace=0.35)

# Fila 1: Espacios geométricos
datasets = [X, X_mm, X_z]
titulos = [
    "Datos originales\n(dominio del Salario)",
    "Min-Max\n(espacio [0, 1])",
    "Z-score\n(media = 0, \u03c3 = 1)",
]
ejes_x = ["Salario (MXN)", "Salario normalizado", "Salario (z)"]
ejes_y = ["Edad (años)", "Edad normalizada", "Edad (z)"]

for col, (Xd, tit, ex, ey) in enumerate(zip(datasets, titulos, ejes_x, ejes_y)):
    ax = fig.add_subplot(gs[0, col])
    for k, p in enumerate(personas):
        ax.scatter(Xd[k, 0], Xd[k, 1], color=colores[k], s=110, zorder=3)
        ax.annotate(p, (Xd[k, 0], Xd[k, 1]),
                    textcoords="offset points", xytext=(8, 5), fontsize=11,
                    color=colores[k], fontweight="bold")
    ax.set_title(tit)
    ax.set_xlabel(ex)
    ax.set_ylabel(ey)
    ax.grid(True, linestyle="--", alpha=0.5)

# Fila 2 izquierda: distancias escaladas
ax_bar = fig.add_subplot(gs[1, :2])
b1 = ax_bar.bar(x_pos - ancho/2, mm_dist, ancho, label="Min-Max",
                color="#1565C0", alpha=0.85)
b2 = ax_bar.bar(x_pos + ancho/2, z_dist, ancho, label="Z-score",
                color="#E65100", alpha=0.85)
ax_bar.set_xticks(x_pos)
ax_bar.set_xticklabels(etiquetas)
ax_bar.set_ylabel("Distancia euclídea")
ax_bar.set_title("Distancias tras escalamiento (Min-Max vs. Z-score)")
ax_bar.legend(frameon=False)
ax_bar.bar_label(b1, fmt="%.4f", padding=3, fontsize=8)
ax_bar.bar_label(b2, fmt="%.4f", padding=3, fontsize=8)
ax_bar.set_ylim(0, max(z_dist) * 1.3)

```

```

# Fila 2 derecha: contribución porcentual
ax_pct = fig.add_subplot(gs[1, 2])
bar_s = ax_pct.bar(etiquetas, pct_sal, label="Salario",
                  color="#B71C1C", alpha=0.85)
bar_e = ax_pct.bar(etiquetas, pct_edad, bottom=pct_sal, label="Edad",
                  color="#2E7D32", alpha=0.85)
ax_pct.set_ylabel("Porcentaje (%)")
ax_pct.set_title("Contribución por atributo\n(sin escalamiento)")
ax_pct.yaxis.set_major_formatter(mticker.PercentFormatter())
ax_pct.set_ylim(0, 115)
ax_pct.legend(frameon=False, fontsize=8)
for bar, pct in zip(bar_e, pct_edad):
    ax_pct.text(bar.get_x() + bar.get_width()/2,
                bar.get_y() + bar.get_height() + 1,
                f"{pct:.1f} %", ha="center", va="bottom",
                fontsize=8, color="#2E7D32")

plt.suptitle(
    "Escalamiento y Normalización: Transformación del Espacio Geométrico",
    fontsize=13, fontweight="bold", y=1.01
)
plt.show()

```

Escalamiento y Normalización: Transformación del Espacio Geométrico

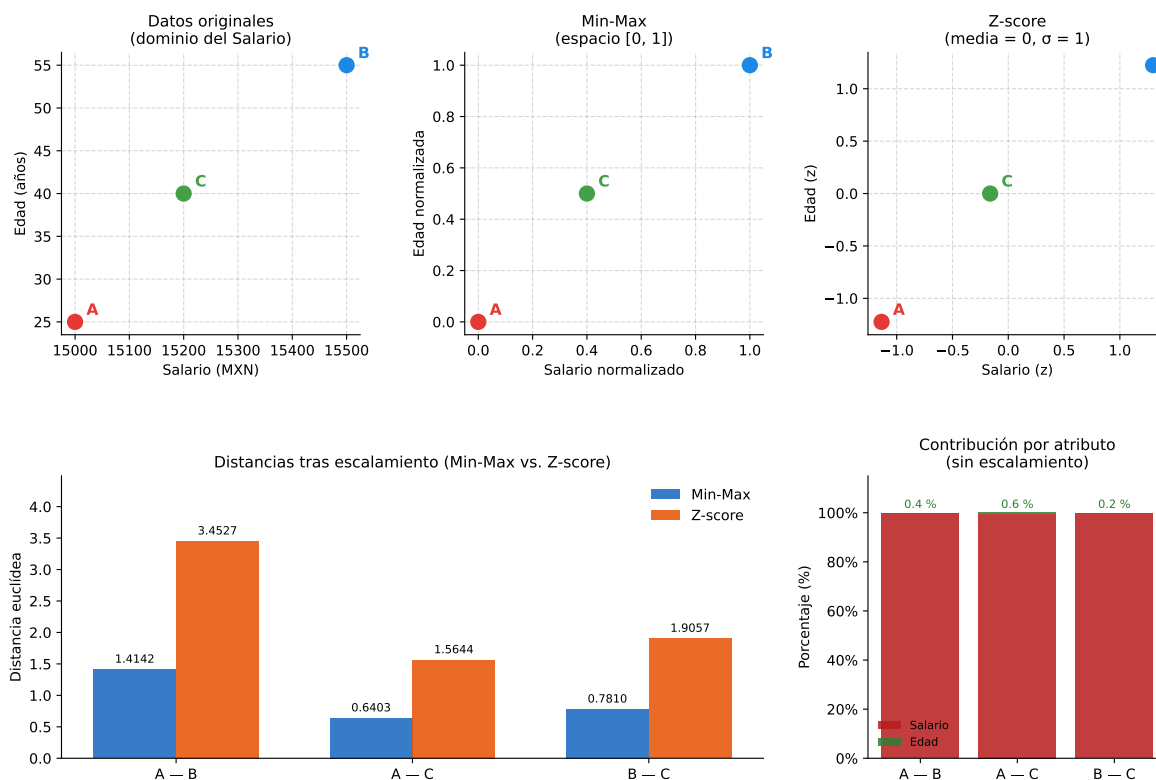


Figure 8.1: Fila superior: transformación del espacio geométrico en los tres escenarios. Fila inferior: distancias escaladas entre pares y contribución porcentual por atributo sin escalamiento.

8.3.3 Interpretación

La **fila superior** de la figura muestra cómo cambia el espacio geométrico en cada escenario:

- **Original.** El eje del salario domina completamente. La distribución de los puntos refleja casi exclusivamente las diferencias en ingresos; la edad es invisible en la geometría del espacio.
- **Tras Min-Max.** Ambos ejes quedan restringidos a $[0, 1]$. Los tres puntos ahora ocupan simétricamente el espacio y cada atributo contribuye en la misma proporción.
- **Tras Z-score.** Los puntos quedan centrados en el origen. La distancia entre ellos refleja la variabilidad estadística relativa de cada atributo, no su magnitud absoluta.

La **fila inferior** confirma numéricamente lo anterior: sin escalamiento, el salario acapara más del **99 %** de la distancia en todos los pares, convirtiendo a la edad en ruido. Tras cualquier

escalamiento, el orden relativo de cercanía se preserva (A — C es el par más cercano), pero ahora ambos atributos influyen de forma genuina en la geometría del espacio.

8.4 Resumen Comparativo: Min-Max vs. Z-score

Criterio	Min-Max	Z-score
Datos sin valores atípicos	Ideal	Funciona
Datos con valores atípicos	Se distorsiona	Mas robusto
Se requiere rango [0, 1]	Garantizado	No aplica
Algoritmos que asumen normalidad (PCA, SVM)	Aceptable	Preferido
Redes neuronales / deep learning	Muy usado	Valido
Llegada frecuente de nuevos datos	Puede violar el rango	Mas estable

Regla fundamental. Al aplicar técnicas de clustering basadas en distancia (como K-Means o PAM) es obligatorio escalar los datos previamente para evitar que variables con magnitudes numéricas grandes dominen el proceso de agrupamiento.

8.5 Preguntas de Reflexión

1. Si el 99 % de los clientes tiene salarios entre \$10,000 y \$20,000, pero existe un directivo con \$500,000, ¿qué técnica de escalamiento es más apropiada y por qué? ¿Cómo afectaría ese valor extremo a la normalización Min-Max?
2. ¿Es siempre correcto escalar los datos antes de aplicar clustering? ¿Podría existir algún escenario donde escalar los datos sea contraproducente o elimine información valiosa?

3. Supongamos que ajustamos nuestro escalador Min-Max con los datos de entrenamiento actuales, donde el salario máximo registrado es de \$20,000, y el modelo sale a producción. Al mes siguiente, el sistema procesa a un nuevo cliente que gana \$25,000. Si le aplicamos exactamente la misma fórmula Min-Max con los parámetros guardados, ¿qué valor matemático obtendrá este nuevo registro? ¿Qué implicaciones algorítmicas tiene esto y cómo se comporta el Z-score ante el mismo escenario?
-

References

- Codificando Bits. 2023. *Similitud Del Coseno Explicada*. <https://www.youtube.com/watch?v=VzVch8oqtUE>.
- Datacamp. 2024. *Comprender La Distancia Euclidiana: De La Teoría a La Práctica*. <https://www.datacamp.com/es/tutorial/euclidean-distance>.
- Datacamp. 2025. *Explicación de La Distancia Hamming: Teoría y Aplicaciones*. <https://www.datacamp.com/es/tutorial/hamming-distance>.
- Han, J., M. Kamber, and J. Pei. 2011. *Data Mining: Concepts and Techniques*. 3rd ed. Morgan Kaufmann. <https://www.sciencedirect.com/book/9780123814791/data-mining-concepts-and-techniques>.
- Krants, Tom, and Alexandra Jonker. 2024. *Cosine Similarity*. IBM. <https://www.ibm.com/mx-es/think/topics/cosine-similarity>.
- López, A., G. Avilés, and S. López. 2021. *Minería de Datos Con r*. (México), 260–62. <https://tienda.fciencias.unam.mx/es/inicio/1603-mineria-de-datos-con-r-version-pdf-9786073047500.html>.
- Medium. 2026. *Demystifying Manhattan Distance: A Data Scientist's Guide to the L1 Norm*. <https://medium.com/@seofrugalscientific/demystifying-manhattan-distance-a-data-scientists-guide-to-the-l1-norm-39321295e875>.
- Tan, Pang-Ning, Michael Steinbach, and Vipin Kumar. 2014. *Introduction to Data Mining*. 2nd ed. Pearson Education Limited. https://rajeshreddycse.wordpress.com/wp-content/uploads/2023/02/introduction-to-data-mining-1292026154-9781292026152_compress.pdf.